

Минобрнауки России
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа программной инженерии

Презентация на тему:
Сравнительный анализ Jest, Mocha и Vitest
по времени выполнения, CPU и ОЗУ

по дисциплине «Методы тестирования программного обеспечения»

Выполнила студентка группы
з5130903/40002
Фёдорова Ирина Романовна

Руководитель:
старший преподаватель
В. А. Пархоменко

Санкт-Петербург, 2026

Для одного приложения можно использовать разные средства тестирования.

Задача: сравнить Jest, Vitest и Mocha и выбрать подходящий инструмент.

- Все выполняют одинаковые 99 тестов.
- Сравниваются время выполнения, работа процессора и память.
- Результаты учитываются только при успешном прохождении всех тестов.

Что делает приложение

Программа находит самый длинный палиндром, его длину и число палиндромных вхождений.

```
npm run start -- "abba"
```

```
npm run start -- --file data/example.json
```

Входной файл содержит строки в формате JSON:

```
{"strings": ["abba", "топот", ""]}
```

Пример: для abba ответ — abba, длина 4, вхождений 6.

Палиндром читается одинаково слева направо и справа налево.

- 1 Между символами добавляются разделители, чтобы одинаково обрабатывать чётные и нечётные палиндромы.
- 2 Для каждой позиции определяется, насколько далеко палиндром продолжается в обе стороны.
- 3 Уже найденная симметрия помогает избежать повторных сравнений.

Пример abba: четыре отдельных символа, bb и abba. Всего 6 вхождений.

Время и дополнительная память растут линейно с длиной строки: $O(n)$.

Для чего нужны Jest, Vitest и Mocha

Все три инструмента запускают тесты и показывают найденные ошибки.

Jest

Готовый набор средств для проверки функций, серверного кода и компонентов интерфейса JavaScript-приложений.

Vitest

Подходит для JavaScript и TypeScript. Удобен в проектах с Vite, потому что может использовать настройки сборки приложения. Работает и с Node.js.

Mocha

Запускает тесты JavaScript-программ. Средства сравнения правильных и полученных ответов выбираются отдельно.

Что проверяют тесты

- Правильный ответ для обычных строк, пустой строки и повторяющихся символов.
- Работу с русскими буквами, пробелами и эмодзи.
- Чтение JSON и понятные сообщения при неверных данных.
- Запуск программы через консоль и правильный вывод результата.
- Работу на длинных строках.

В каждом фреймворке — одни и те же 99 тестов.

Для 511 коротких строк ответ также сравнивается с независимым простым перебором.

По четыре файла: алгоритм, JSON, консоль и нагрузка.

Запуск всех тестов: `npm run test`.

Главные проверки: ответ и ошибка во входе

```
test('Чётная длина', () => {  
  assert.deepEqual(manacher('abba'), {  
    longest: 'abba', length: 4, count: 6,  
  });  
});
```

```
test('Число внутри пакета', () => {  
  assert.throws(  
    () => parseInput('{"strings":[1]}'), /Элемент 1/  
  );  
});
```

`deepEqual` сравнивает весь ответ с правильным.

`throws` проверяет, что неверные данные вызывают ожидаемую ошибку.

Проверка программы через консоль

```
test('Консоль обрабатывает строку', () => {  
  const result = run(['abba']);  
  assert.equal(result.status, 0);  
  assert.match(result.stdout,  
    /Самый длинный палиндром: "abba"/);  
  assert.match(result.stdout, /вхождений: 6/);  
  assert.equal(result.stderr, '');  
});
```

- run запускает настоящую консольную программу.
- Код 0 означает успешное завершение.
- Тест проверяет ответ и отсутствие сообщений об ошибке.

Как проверялось качество тестов

В папке `mutations` хранятся три примера ошибок. Скрипт проверяет каждый во временной копии программы:

- ❶ Выбор правого палиндрома вместо левого при равной длине.
- ❷ Неправильное количество палиндромных вхождений.
- ❸ Удаление пробелов из входных строк.

Все три фреймворка обнаружили каждую ошибку.

Это подтверждает, что соответствующие тесты замечают неправильное поведение программы.

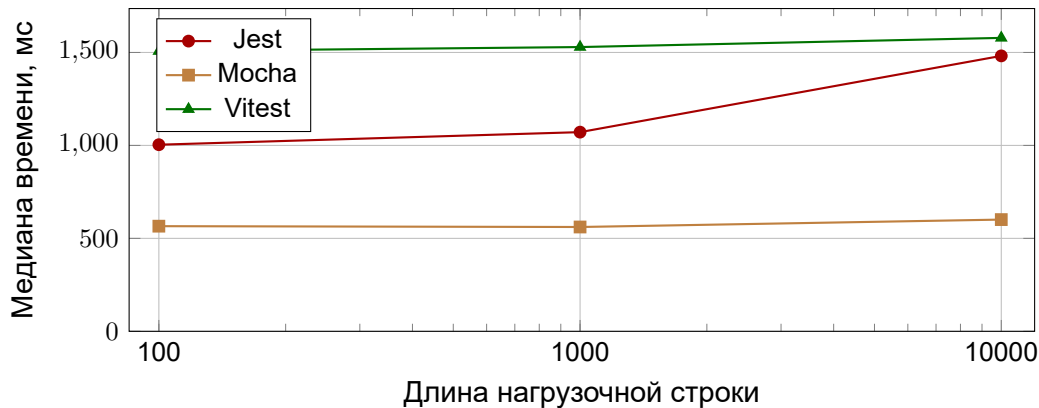
Как проводилось сравнение

- На одном компьютере по очереди запускались все 99 тестов.
- Использовались строки длиной 100, 1000 и 10000 символов.
- Для каждого размера и фреймворка выполнено 7 измерений.
- Всего получено 63 успешных запуска. Первые пробные запуски не учитывались.
- Отдельная программа измеряла время, работу процессора и память всех обнаруженных процессов тестового запуска.

Команды: `npm run benchmark` и `npm run results`.

В таблицах и на графике показана медиана: среднее по позиции значение среди семи результатов, упорядоченных по величине.

Сколько времени занимают тесты



Каждый результат – медиана семи запусков всех 99 тестов.

Время, процессор и память

Строки длиной 10000 символов. Медианы семи запусков.

Раннер	Время, мс	CPU, мс	RSS, MiB
Jest	1481.6	1515.6	132.2
Mocha	601.0	484.4	101.1
Vitest	1578.4	1812.5	170.2

- Время: сколько длится весь запуск тестов.
- CPU: время работы процессора над тестами.
- RSS: наибольшая замеченная сумма памяти процессов.

Короткие процессы и пики между опросами могут быть пропущены. CPU-время является нижней оценкой.

Для этого приложения выбран Mosha.

- Все три фреймворка успешно выполнили 99 тестов.
- Mosha показал лучшие результаты по времени, CPU и памяти на всех трёх размерах входа.
- Каждый параметр имел одинаковый вес при выборе.

Вывод относится к этому приложению, компьютеру и настройкам. Для другого проекта измерения нужно повторить.